

Modelado y Programación

Proyecto 02 : Índice de cobertura nubosa

Descripción del proyecto

Este proyecto se centra en calcular el Índice de Cobertura Nubosa (CCI) a partir de imágenes capturadas por una estación de registro atmosférico. La estación toma fotografías del cielo cada 15 minutos utilizando una lente gran angular de 360 grados. Las imágenes se almacenan en formato JPEG y se desea determinar la proporción de la bóveda celeste cubierta por nubes. Para ello, se utilizara el CCI, que representa el cociente entre el número de píxeles clasificados como nube y el número total de píxeles en el área de interés de la imagen. El objetivo del proyecto es analizar las imágenes y calcular el CCI para comprender el nivel de cobertura nubosa en cada fotografía capturada.

Para resolver la tarea, nos basaremos en el proceso de solución de problemas visto en clase. A continuación, se mostrara nuestro análisis del mismo, la plantación de la solución (el algoritmo y herramientas que usaremos), entre otras cosas como los reportes de las juntas.

Entrada del programa

El programa recibe como entrada el nombre de un archivo de imagen en formato **JPEG** con las siguientes características:

- Ancho: 4,368 píxeles
- Alto: 2,912 píxeles
- Centro de la imagen: Columna 2184 y Renglón 1456
- Tamaño del archivo: Alrededor de 32MB
- El área de interés es un círculo de radio aproximado de 1324 píxeles contenido en el rectángulo de la imagen. Cada píxel tiene tres componentes de color: R (rojo), G (verde) y B (azul).
- El programa también puede recibir una bandera "S" (mayúscula o minúscula) como parámetro opcional. Si se proporciona esta bandera, el programa generará una imagen en blanco y negro con las nubes en blanco y el cielo en negro.

Salida del programa

El programa desplegará en la salida estándar el índice de cobertura nubosa (CCI), calculado como el cociente entre el número de píxeles clasificados como "nube" el número total de píxeles en el área de interés de la imagen.

Si se invoca al programa con la bandera "S", también generará una imagen en formato JPEG o PNG en blanco y negro con las nubes en blanco y el cielo en negro. El nombre del archivo de imagen de salida será similar al del archivo de entrada, pero con el sufijo "-seg" añadido antes de la extensión.

Requisitos funcionales

Los requisitos funcionales son aquellos que se refieren a lo que debe hacer el programa como tal. Los que consideramos como requisitos funcionales para este proyecto son los siguientes:

- El programa debe aceptar como entrada el nombre de uno o más archivos de imagen en formato JPEG con las características especificadas en la entrada del programa.

- Debe ser capaz de manejar la opción de la bandera "S" para generar la imagen en blanco y negro. La funcionalidad de la bandera deberá ser la de generar y guardar una imagen en blanco y negro con las nubes en blanco y el cielo en negro.
- La salida estándar del programa deberá ser el valor del CCI de las imágenes en un tiempo aceptable.
- El programa debe clasificar los píxeles como nube o cielo basándose en ciertos criterios de color (RGB).
- El programa deberá contar el número de píxeles clasificados como nube y calcular el CCI como el cociente entre estos píxeles y el número total de píxeles en el área de interés (el círculo de la imagen).

Requisitos no-funcionales

Los requisitos no-funcionales son aquellos que se refieren a cómo debe comportarse el programa. Lo que esperamos lograr en el proyecto es lo siguiente:

Eficiencia: El programa debe ser constante en el manejo de imágenes, especialmente para imágenes grandes, considerando la restricción de tiempo para el procesamiento. Además de que no deberá gastar recursos extra para esa tarea.

Tolerancia a fallas: El programa debe detectar posibles fallas como la ausencia del archivo de entrada, errores de lectura de imagen, y otros problemas que puedan surgir durante la ejecución y manejarlos o erradicarlos.

Amigabilidad: En este proyecto el grupo de usuarios al que esta dirigido debe ser capaz de manejar el programa en terminal de la forma más intuitiva posible.

Escalabilidad: El programa deberá poder ser capaz de poder actualizarse de forma relativa sencilla a futuro.

Seguridad: El programa no debe realizar acciones maliciosas ni comprometer la seguridad del sistema en el que se ejecuta. Además, de ser necesario deberá poder proteger los datos privados.

Herramientas

Se decidió que se utilizará el lenguaje de programación Python, y por consecuencia su librería estándar `argparse` y la librería `pillow`.

Python es ampliamente utilizado en la comunidad científica y de desarrollo debido a su sintaxis clara y legible. Además, cuenta con una gran cantidad de librerías, como `pillow` para el procesamiento de imágenes mientras que `argparse` facilita el análisis de argumentos de línea de comandos, lo cual permitirá una interfaz amigable para los usuarios.

Para el trabajo cooperativo se utilizara git y GitHub. Esto debido a que permiten trabajar de forma eficiente en el código, realizar un seguimiento de los cambios, gestionar problemas y realizar revisiones de código.

Pseudo-código

`main.py`

```
import argparse
from cloud import CloudImage
```

```

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("images", help="archivos de imagenes", nargs='+' type=Path)
    parser.add_argument("-s", "--save", help="escribir archivos procesados con sufijo -seg",
        action=argparse.BooleanOptionalAction)
    args = parser.parse_args()
    images = []

    for file in args.images:
        with open(file) as img:
            images.append(image.CloudImage(img))

    for img in images:
        if args.save:
            img.write_conv()
            print(f"{img.name}: {img.cloud_cov}")

```

cloud.py

```

from PIL import Image
from pathlib import Path

class CloudImage:
    def __init__(self, fname):
        path = Path(fname)
        self.name = path.stem
        self.suffix = path.suffix

    def cloud_cov(self):
        sky = 0
        cloud = 0
        with Image.open(self.name) as img:
            pixels = img.load()
            width, height = img.size
            for x, y in zip(range(width), range(height)):
                if pixels[x, y].hsv > HSV_UPPER_THRESHOLD:
                    sky += 1
                elif pixels[x, y].hsv < HSV_LOWER_THRESHOLD:
                    cloud += 1

        return (sky * 100) / (width * height)

    def write_conv():
        with Image.open(self.name) as img:
            pixels = img.load()
            width, height = img.size
            out = Image.new(img.mode, img.size)
            pixels_out = out.load()
            for x, y in zip(range(width), range(height)):
                if pixels[x, y].hsv < HSV_LOWER_THRESHOLD:

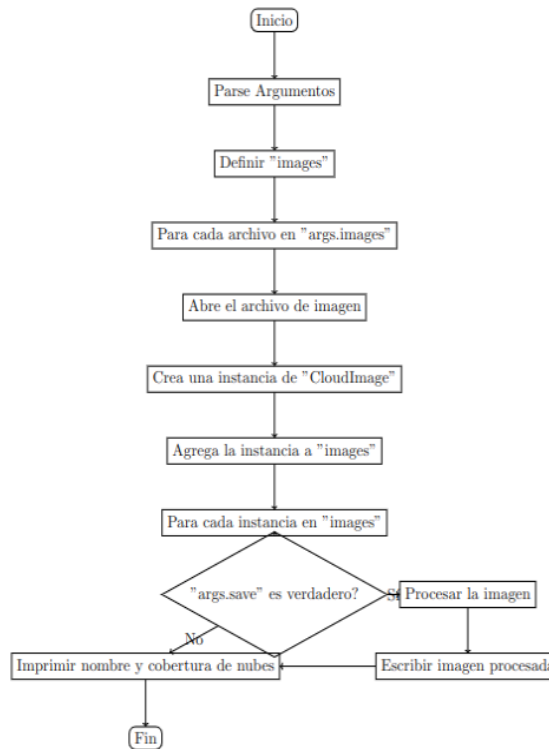
```

```

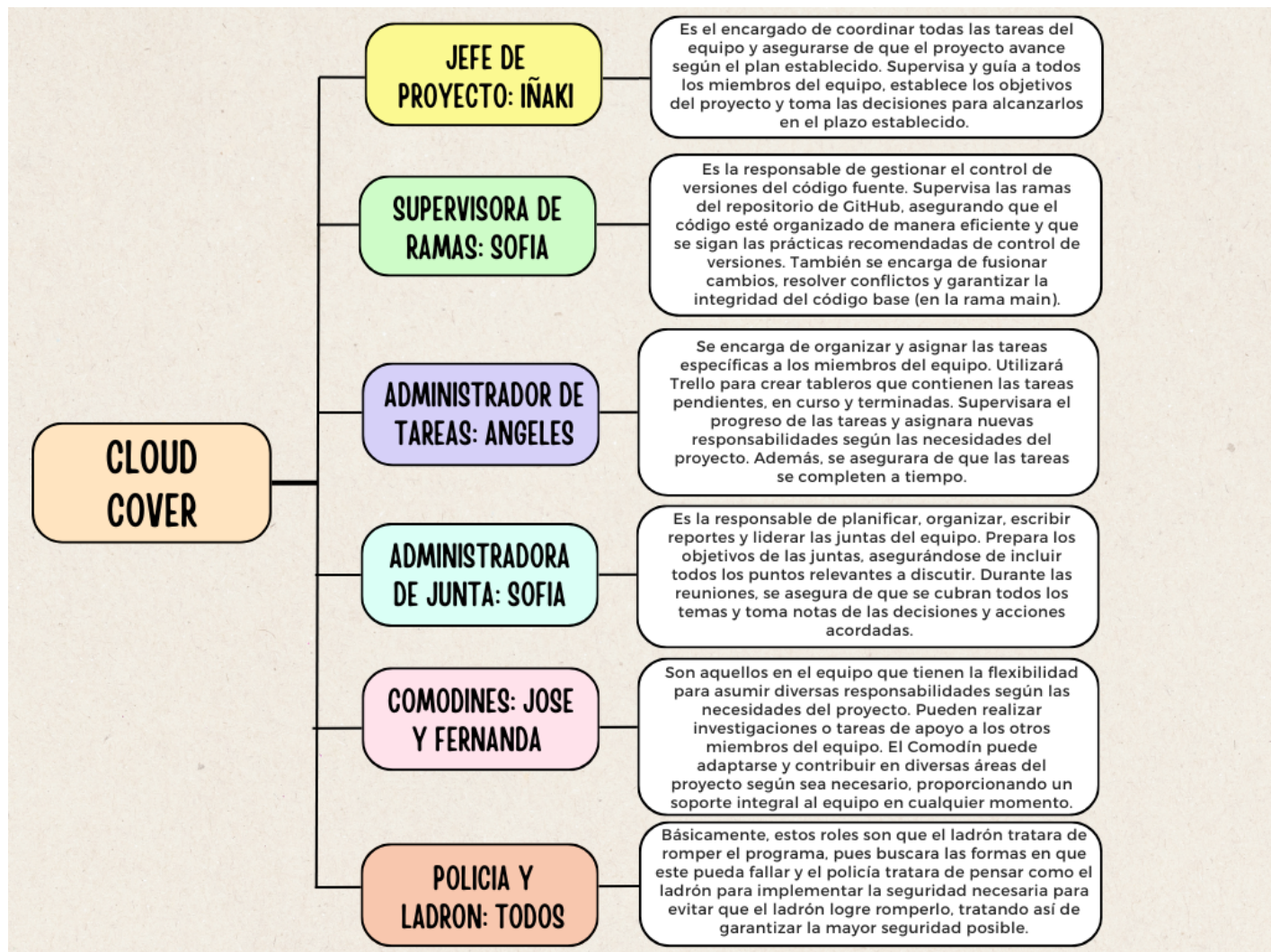
        pixels_out[x, y] = (255, 255, 255)
    else:
        pixels_out[x, y] = (0, 0, 0)
    out.save()
    out.close(self.name + "-seg" + self.suffix)

```

Diagrama de flujo



Jerarquía de roles



Resumen de juntas

Junta 1

La primer junta se realizo el lunes, 23 de Octubre de 2:00pm a 3:00pm. En el preparación de la administradora de juntas, ella decidió que estos serían los puntos a tocar en la junta:

- Repaso y explicación de roles.
- Que posición (rol) tendrá cada integrante del equipo.
- Asignación de tareas y tiempos (nota de la administradora de juntas: pensar en dos tipos de tareas: de roles y de código).

A continuación se mostrara el resumen de la junta:

2:00pm - Inicia la junta 1.

2:00pm - 2:25pm

Jefe de proyecto ayuda a crear SSH para poder mejorar en el manejo de git debido a que se considera necesario para trabajar de forma mejor. Al final no se termino en la junta la creación de SSH, y por complicaciones de hardware la administradora de ramas utilizara por el momento HTTPS. Se decide que se revisara el tema por individual para que cada quien pueda ver que le conviene.

2:26pm - 2:34pm

Se explican brevemente los roles con los integrantes presentes en la junta.

2:35pm - 2:43pm

Se asignan los roles del equipo con los integrantes del equipo presentes:

- Jefe de proyecto: Iñaki
- Supervisor de ramas: Sofía
- Administrador de tareas: Angeles → buscar trello u otras herramienta para organizar las tareas de todos
- Administrador de junta: Sofía → escribir reporte
- Comodín: José Pablo

Nota: Recordar como manejaremos a los miembros del equipo ausentes en la junta.

2:44pm - 2:56pm

Discusión de herramientas:

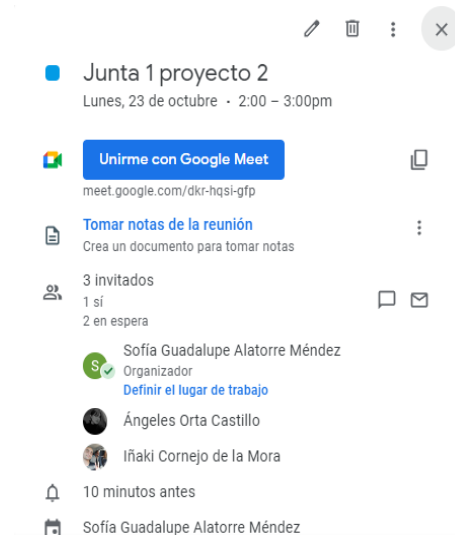
Se decidió que se usara python, no se usara pillow, ni SQL.

Se usara OpenCV (en la junta posterior esto cambia, pero por el momento se decidió eso).

2:56pm - Se cierra la junta 1.

Anotaciones técnicas de la junta 1:

La misma se realizó de forma presencial con los miembros disponibles del equipo a esa hora. Además, si se registro en Meet la misma, se enviaron invitaciones y si bien no aparecen aceptadas en la captura de pantalla debido a que fue en presencial, la administradora de junta se metió con el propósito de que se registrara la junta.



Junta 2

La segunda junta se realizó el miércoles, 25 de Octubre de 11:30am a 11:54am. En preparación para la junta, la administradora de juntas decidió que estos serían los objetivos de la junta:

- Asignar los dos nuevos roles a miembros del equipo (policía y ladrón)
- Ver el proceso de solución del problema:
 - Descripción del problema ✓
 - Entrada ✓
 - Salida ✓
 - Análisis y la idea ✓
 - Pseudocódigo - en bosquejo
 - Diagrama de flujo - en bosquejo
 - Herramientas - elegir el arsenal ✓ (Se eligió el lenguaje de programación que será Python y se eligió la librería a usar)
 - ¿Pruebas? → No se decidió.

A continuación se mostrara el resumen de la junta:

11:27am - Inicia la junta 2.

11:27am - 11:31am

Tras una consideración que se realizó fuera de las juntas, se decide que se usara Pillow en vez de OpenCV debido se consideraron las restricciones de tiempo y recursos del equipo. Además, dada la naturaleza del problema y el alcance del proyecto, se determinó que la funcionalidad requerida para el cálculo del Índice de

Cobertura Nubosa podía implementarse de manera adecuada y eficiente utilizando la biblioteca Pillow en lugar de OpenCV.

11:32am - 11:40am

Hablamos con el comodín del equipo cuales podrían ser sus tareas y realizamos una discusión en equipo de las responsabilidades del equipo como tal.

11:41am - 11:50am

Se realizo una lluvia de ideas para pseudocódigo y una pequeña idea de como sería el pseudocódigo.

1 recibe la imagen

2 lee la imagen color a color

- Pillow es lento por eso

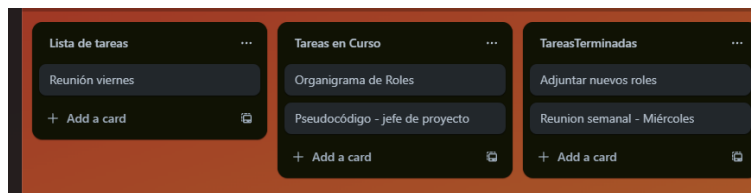
2.5 leer

...

El jefe de proyecto seguirá investigando para revisando eso, aunque como equipo también buscaremos como podemos ignorar.^{el} espacio fuera del circulo del lente de la cámara para calcular el Índice de Cobertura Nuboso.

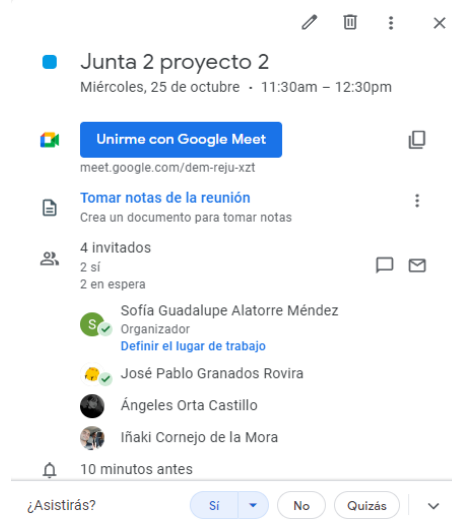
11:51am - 11:53am

Conclusión de la junta: Se decidió que todo el equipo se encargará de ser policías y ladrones, además, se asignaron más tareas que la administradora de tareas anotó en trello:



Anotaciones técnicas de la junta 2:

La misma se realizo de forma semi-presencial con dos miembros del equipo conectados al Meet (la administradora de juntas y el comodín), el comodín se conecto de forma virtual y la administradora de juntas prendió su cámara para que la junta fuera lo más fluida posible con todos los miembros del equipo. Se registro la reunión en Meet, aunque sólo aparecen dos invitaciones aceptadas, si participo todo el equipo.



Junta 3

La tercer junta se realizo el viernes, 27 de Octubre de 10:30am a 11:10am. En preparación para la junta, la administradora de juntas decidió que estos serían los objetivos principales de la junta:

- Seguir trabajando en las ideas y análisis para el pseudocódigo y diagrama de flujo del proyecto. ✓
- Continuar con los puntos que faltan del proceso de solución de problemas:
 - Requisitos funcionales - mencionarlos ✓
 - Requisitos no-funcionales - mencionarlos ✓
 - Hacer sección en el pdf de las herramientas y su explicación/justificación del uso de ellas ✓
 - La sección de escatimar trabajo inútil se realiza mediante la discusión en las juntas cuando hablamos de la forma óptima del trabajo
- Checar con el nuevo miembro del equipo que se integro cual será su posición en el equipo y resumirle lo visto en juntas anteriores ✓

A continuación se mostrara el resumen de la junta:

10:30am - Inicia la junta 3.

10:33am - 10:36am

Se inicia una breve discusión del pseudocódigo, el jefe del proyecto se encarga de escribirlo. Lo mandara al finalizar la junta.

10:36am - 10:51am

Se integra el ultimo miembro del equipo, se discute que rol tomara y se resumen las juntas anteriores.

Como el uso de las herramientas trello, roles, se ve que rol puede tomar el nuevo miembro.

Se decide que Fernanda también sera comodín junto con José Pablo, pero como tal todo mundo estará involucrado en todo. También podría tomar rol de administradora de ramas y las tareas que pueden compartir.

10:40am - 10:44am

Mientras también se discuten las herramientas, las cuales serán: python, argparse (librería estandar), pillow y github.

10:53am - 11:05am

El jefe de proyecto explica el pseudocódigo, lo siguiente es un breve resumen de lo que pudo escribir la administradora de juntas, el pseudocódigo se encuentra en limpio en la sección de pseudocódigo del documento: Para el problema sólo se necesitan dos archivos.

1. Lo primero es el entrypoint, para checar argumentos de forma mas elegante.
2. Se crear un parser para checar la lista de los archivos.
3. Se itera a través de los archivos y se abren individualmente.
4. Se agregan como objetos al objeto imagen.
5. Se tiene la lista de imágenes, se recorre, se llama el método que da el índice de cobertura nubosa y se imprime.
6. Checamos si se dio con **S** el argumento y vemos si se imprime o no, se habla que las tecnicidades se verán tras tener las imágenes.

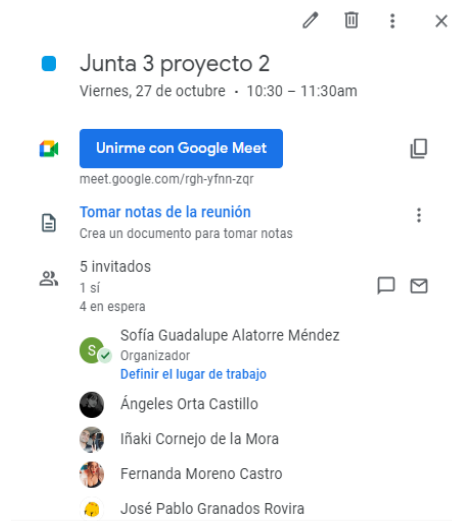
11:06am - 11:09am

Conclusión de la junta: Se vio el pseudocódigo y se termino de asignar roles y tareas (las tareas con sujeto a cambio del avance del proyecto).

11:10am - Se cierra la junta 3.

Anotaciones técnicas de la junta 3:

La misma se realizo de forma semi-presencial, ya que el nuevo miembro del equipo se integro mediante Meet a la misma, de nuevo, la administradora de juntas decidió prender la cámara para que hubiera mejor fluidez e interacción con todos los miembros del equipo. Además, si se registro en Meet la misma, se enviaron invitaciones y si bien no aparecen aceptadas en la captura de pantalla debido a que fue en presencial, la administradora de junta se metió con el propósito de que se registrara la junta.

**Junta 4**

La cuarta junta se realizo el sábado, 4 de Noviembre de 2:10pm a 3:06pm. En preparación para la junta, la administradora de juntas decidió que estos serían los objetivos principales de la junta:

- Repartir responsabilidades y tareas en el código, algunas de las ideas que se le ocurrió fueron las siguientes:
 - Encargado del cloud.py (Procesamiento de imágenes)
 - Encargado de pruebas (pruebas unitarias)
 - Encargado de ramas (Control de versiones, Pull requests)
 - Encargado de optimización del codigo Encargado de la linea de comandos

Hubo una discusión con el jefe de proyecto y la administradora de tareas para revisar las sugerencias y se llegó a una estructura más detallada para la distribución de las tareas, que incluyó aspectos como manejo de la línea de comandos, procesamiento directo de imágenes en cloud.py, algoritmo de clasificación en cloud.py, el testbench para correr pruebas y la escritura de pruebas por parte de cada miembro para sus respectivas tareas. Además de unas sugerencias de quien podría encargarse de cada una de las tareas.

A continuación se mostrara el resumen de la junta:

2:10pm - Inicia la junta 4.

2:10pm - 2:23pm

Se inicio la junta a pesar de la ausencia del jefe del proyecto y el comodín. El resto del equipo decidió iniciar la junta para no retrasarse. Se discutió el resumen de la conversación del administrador de juntas con los otros dos miembros (la administradora de tareas y el jefe de proyecto) acerca de las tareas y se acordó que se podrían comenzar a trabajar en las tareas asignadas, en especial se tomo enfoque en las habilidades de los comodines, quienes podría apoyar en cualquier tarea o incluso asumir una tarea completamente diferente por si mismos dependiendo de lo que se necesite.

2:24pm - 2:28pm

Se decide discutir los detalles específicos de cada tarea, empezando con la tarea de líneas de comandos, se estableció que implicaba la verificación de errores, el manejo de una o más imágenes y la implementación de la bandera -S junto con las funciones de argumentos correspondientes.

2:29pm - 2:39pm

Para la tarea de manejo directo de la imágenes en cloud.py, se detallaron las funciones necesarias, incluyendo cargar imágenes, clasificar píxeles en blanco y negro y calcular índices de clasificación. Las funciones mencionadas fueron: cargar imágenes, clasificar, imprimir en blanco y negro, calcular el índice y función de impresión.

2:40pm - 2:43pm

La tarea de algoritmo de clasificación se definió como relacionada a la tarea anterior, pero responsable en específico de las funciones de clasificación en blanco y negro y operaciones similares.

2:44pm - 2:56pm

Se discutieron al menos cinco pruebas que debían realizarse, que incluían cargar imágenes, clasificar en blanco y negro, calcular el índice de cobertura y probar los argumentos.

2:57pm - 3:01pm

Se asignaron supervisores para cada una de las tareas, las cuales quedaron así:

- Línea de comandos: Sofía, fernanda y Angeles.
- Algoritmo de clasificación: Iñaki.
- Manejo directo de la imagen: José.
- Todo mundo hará sus respectivos tests.

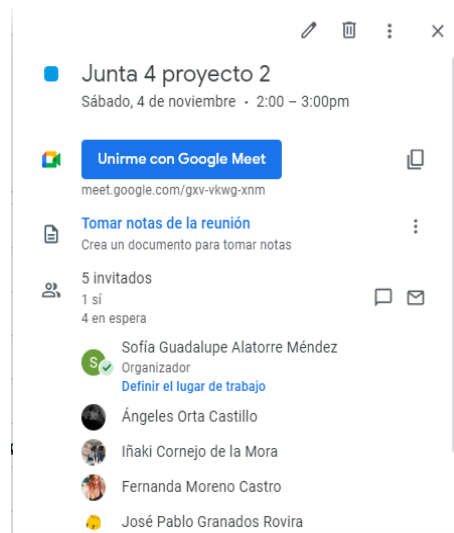
3:02pm - 3:05pm

Se hablo cómo se podrían gestionar las ramas del proyecto mediante comentarios y las posibles ramas necesarias para cada una.

3:06pm - Se cierra la junta 4.

Anotaciones técnicas de la junta 3:

La misma se realizo de forma virtual, debido a que era fin de semana y no era posible juntarse en presencial.



Fuentes utilizadas:

- 1) <https://neptune.ai/blog/image-processing-python>